Errand Agent — design note

Shape. Single agent; the LLM *understands and plans*, the application *decides and acts*. The model sees 10 tools (`update_request`, `search_providers`, `get_provider`, `list_bookings`, `propose_booking|reschedule|cancel`, `remember`, `forget`, `show_memories`). `hold_slot`, `book`, `cancel`, `reschedule` are **not exposed to the model at all** — they run only inside the app's executor. Trade-off: less model autonomy, but no prompt (or injected review) can produce a side effect. One agent suffices; a second "verifier" agent would add latency without adding a guarantee that code doesn't already give.

State model. Explicit `Workflow` per session (`state.py`): `IDLE -> GATHERING -> SEARCHING -> AWAITING_APPROVAL -> EXECUTING -> VERIFYING -> DONE|FAILED`, with an allowed-transition table (illegal moves raise). The `Request` keeps per-field provenance (`user` / `memory:<id>` / `event`); a correction rewrites one field and invalidates any pending proposal, leaving the rest intact. Everything durable (workflow, proposals, approvals, intent ledger, memories, messages, outbox) is in SQLite with autocommit.

Approval enforcement (code, not model). `propose_*` -> the app holds the slot and writes a *proposal* with material fields (provider, slot, **quoted** price / booking id) and a SHA-256 of them. The app — not the model — renders the confirmation card. Only a deterministic parse of the user's *next* message (`classify_reply`: plain "yes/confirm/ok", no negation, no extra instructions, optional matching ref code) creates an approval row. Executors call `authorize(proposal, current_material)` immediately before the side effect; if a re-hold changed price or slot the hash differs -> approval invalidated -> new card. Approvals are single-use. A claim guard rewrites any model text that says "booked/confirmed" unless the executor verified it this turn.

Idempotency & recovery. One intent row per target (`user|book|provider|slot`) -> one idempotency key, **written** `in_flight` **before** `book()` is sent. Repeated "yes", retries and restarts reuse it; a succeeded intent short-circuits to "already booked". `book()` is never blindly retried: on timeout/5xx the app polls `get_booking_status(idempotency_key)` (bounded) -> `confirmed` => success; `not_found` => retry with the *same* key; still unknown => "NOT confirmed", ledger `unknown`, re-checked at the start of every later turn. On startup the agent reconciles `in_flight` intents; if nothing was booked it invalidates the approval and asks again (never resumes a side effect on pre-crash consent). `cancel` 503s are retried by the gateway with the same key.

Bounded execution. Per turn: <=8 model calls, <=14 tool calls, <=60 s wall clock (injectable clock), <=3 transient retries with backoff, <=4 status polls. Every exit — limit, tool failure, unexpected exception, model outage — returns a user-facing message stating what was and wasn't done; the run always ends with a `run_end` event.

Memory & context. Memories carry `source` (explicit|inferred), `confirmed`, `status` (active|superseded|forgotten|rejected). Explicit writes require an `evidence` quote found verbatim in the user's latest message (so reviews can't write memory or change the request). Inferred facts (e.g. "not before 11 am" seen in a request) are stored unconfirmed, **never sent to the model**, and confirmed via an app-rendered question. Forget redacts the row and scrubs value, original statement and distinctive tokens from stored history. `ContextBuilder` enforces a fixed token budget on the real payload: system + state + current turn + this run's tool results (compressed if needed), then eligible memories ranked by relevance to the current service (<=35 % of the remainder), then newest history until full. Every model call logs a manifest: tokens per section, memory ids included and *why each other id was excluded*, history ids kept/dropped, payload hash.

Observability. One JSONL trace per run: state transitions, context manifests, model calls (latency, tokens, est. cost at Sonnet 5.5 $2/$10 per MTok), tool calls (args, sanitized results, latency, attempt), approvals, ledger writes, blocked actions, injection flags. Excerpt from `traces/timeout_after_success.jsonl` (run 2):

```
 3  approval_granted   PRP-0001 by=user_message turn=2
 4  ledger             fd18...-g0 prepared            <- key exists before any side effect
 5  approval_checked   ok (hash of provider/slot/Rs 1250 unchanged)
 7  ledger             in_flight attempts=1         <- durable before book()
 8  tool_call book     ERROR timeout  8600ms        <- ambiguous: may have booked
 9  STATE EXECUTING -> VERIFYING (consulting source of truth)
10 tool_call get_booking_status(key) -> confirmed BK-2002   <- no blind retry
12 booking_verified   verified_by=get_booking_status
14 run_end booked     "[ok] Booked and verified ... it went through exactly once"
```

Evals. 34 scenarios with simulated users against an offline `RuleModel` (or a recorded cassette, replayed strictly by request hash); every scenario also asserts 7 global invariants (no booking/cancel/reschedule without a matching approval, no duplicates, no unverified success claims, all runs terminate, context within budget with no unconfirmed/forgotten facts). Mutation tests prove the invariants fail when the gate is bypassed.